



Modul 3 – Simple Image Operations

A. OBJECTIVES

1. Students can understand and implement Brightness Linear Transformation
2. Students can understand and implement Contrast Image
3. Students can understand and implement Inverse Image
4. Students can understand and implement Logarithmic Brightness Transformation
5. Students can understand and implement the types of Grayscale operations
6. Students can create Gamma Correction applications
7. Students can create image simulations with specified image depth.
8. Students can do denoising by using Averaging
9. Students can perform image masking using logical operators.

B. TOOLS AND MATERIALS

1. PC/Laptop
2. GitHub
3. *Google Collaborator*

C. THEORETICAL REVIEW

C1. Simple Image Operations

a) Inverse Image

Image inversion is a process used to reverse the values of an image by mirroring the center pixel value. The resulting image is known as a negative image. This process is also very simple, as it simply subtracts the red, green, and blue pixel values from 255. $g(x) = 255 - f(x)$, where $g(x,y)$ is the negative image, and $f(x,y)$ is the original image. So if the original pixel value is 255, with this operation the result will be 0 (because $255 - 255$).

b) Brightness Linear Transformation

The Brightness Linear Transform is the simplest digital image processing operation. This operation adds pixel values when the brightness value is increased, and subtracts pixel values when the brightness value is decreased.

$g(x,y) = f(x,y) + b$, where $g(x,y)$ is the pixel value after transformation, $f(x,y)$ is the original pixel value, and b is the brightness value.

Due to the simplicity of this operation, its computation is very fast. Although the computation speed is high, this transformation has a weakness. Linear brightness transformation is a one-way transformation, where the image after being transformed using this operation, cannot be returned to its original value when reverse linear brightness is performed. For example, if the initial pixel value is 240, then transformed by adding a brightness value of 20, then the final pixel value is 260. If the program is run, an error will occur, because the pixel value is only in the range 0 - 255. Likewise, if a subtraction is performed that results less than 0 will also result in an error. To overcome this, usually a Truncate function is created whose task is to prevent the pixel value from being greater than 255 or less than 0. If the value after the operation is greater than 255 then the value will be forced (truncate) to 255, and if the value after the operation is less than 0 then the value will be forced (truncate) to 0. With the Truncate function, the value 240 operated with brightness 20 will become 255 (not 260). If the value 255 is operated with a brightness of -20, it will become 235 (instead of 240), this is what makes the linear brightness transformation irreversible. So this truncate function prevents errors because the pixel value does not fall outside the range of 0-255, but as a result, the pixel that has been brightened cannot be perfectly restored. The solution to this problem can be solved using the Log Brightness transformation, which is also discussed in this module.

The algorithm for truncate can be seen in the following steps:

Steps	Process
1	Start
2	read pixel value f1
3	If $f1 < 0$, then the value of f1 is changed to 0
4	If $f1 > 255$, then the value of f1 is changed to 255
5	Stop

c) Contrast Transformation

Contrast is the degree to which pixels are distributed in terms of color intensity. There are three types of contrast: low contrast, high contrast, and normal contrast.

- Low Contrast Images: Images with low contrast can occur due to a lack of lighting. Images with low contrast have a narrow histogram curve, due to the uneven distribution of light and dark intensity. As a result, the darkest point of

an image does not reach the deepest black and the brightest point does not reach the brightest white.

- High Contrast Image: is the opposite of a low contrast image because it has a wide histogram curve, the distribution of dark and light intensity is even across the intensity scale.
- Normal Contrast Image: Occurs when the width of the histogram curve is not too wide and not too narrow.

Contrast operations can be performed by modifying the formula used in linear brightness, namely:

$g(x,y) = a * f(x,y) + b$, where $g(x,y)$ is the pixel value after transformation, $f(x,y)$ is the original pixel value, a is the contrast value, and b is the brightness value.

In addition to the formula above, contrast transformation can be done by calculating the Contrast Correction Factor using the following formula:

$$F = \frac{259(C + 255)}{255(259 - C)}$$

For this to work, the F value is stored in a double variable, not an integer. The C value represents the desired contrast level.

The next step is to calculate the contrast value for each pixel R, G, and B.

$R' = F(R - 128) + 128$, where R is the Red pixel value and R' is the final Red pixel value after the Contrast operation. Because the result of this operation can be outside the range of 0-255, the truncate function is used. Note that if the truncate function is used, this operation cannot be reversed.

Here is the pseudo-code for the contrast formula above:

1	<code>factor = (259 * (contrast + 255)) / (255 * (259 - contrast))</code>
2	<code>color = GetPixelColour(x, y)</code>
3	<code>redNew = Truncate(factor * (Red(colour) - 128) + 128)</code>
4	<code>new green = Truncate(factor * (Green(colour) - 128) + 128)</code>
5	<code>new blue = Truncate(factor * (Blue(colour) - 128) + 128)</code>
6	<code>SetPixelColour(x, y) = RGB(new red, new green, new blue)</code>

d) Logarithmic Brightness Transformation

Log transformation maps an image with a narrow color *range* to a wider one in the *output image*. *Log transformation is useful in graphing when there are very small differences in values or very large differences in values in a series, so that the very*

small differences in values are difficult to see. A narrow *input of low-level gray values* is mapped into a wide *range of output gray level*.

In general, the form of log transformation is:

$$s = c * \log(1 + r)$$

Where

c : constant

r : grey-level value of the input image

s : grey-level value of the output image

e) **Grayscale**

Grayscale is a grayscale image with an 8-bit pixel depth intensity value. *Grayscale* is usually used to represent light intensity in a single band of the electromagnetic spectrum (infrared, visible light, ultraviolet, etc.). Some frequently used *grayscale methods* are *Averaging*, *Lightness*, and *Luminance*. *Averaging* is determined by averaging the values of each R, G, and B channel. *Lightness* is determined by averaging the maximum and minimum values of all R, G, and B channels. While *Luminance* (Luminosity) is determined by providing a weighting adjusted to the visible spectrum that is most present in natural images.

$$Grayscale_{avg} = \frac{(R + G + B)}{3}$$

$$Grayscale_{Lightness} = \frac{\max[R, G, B] + \min[R, G, B]}{2}$$

$$Grayscale_{Luminance} = 0.21R + 0.72G + 0.07B$$

Original image



Lightness



Average



Luminosity



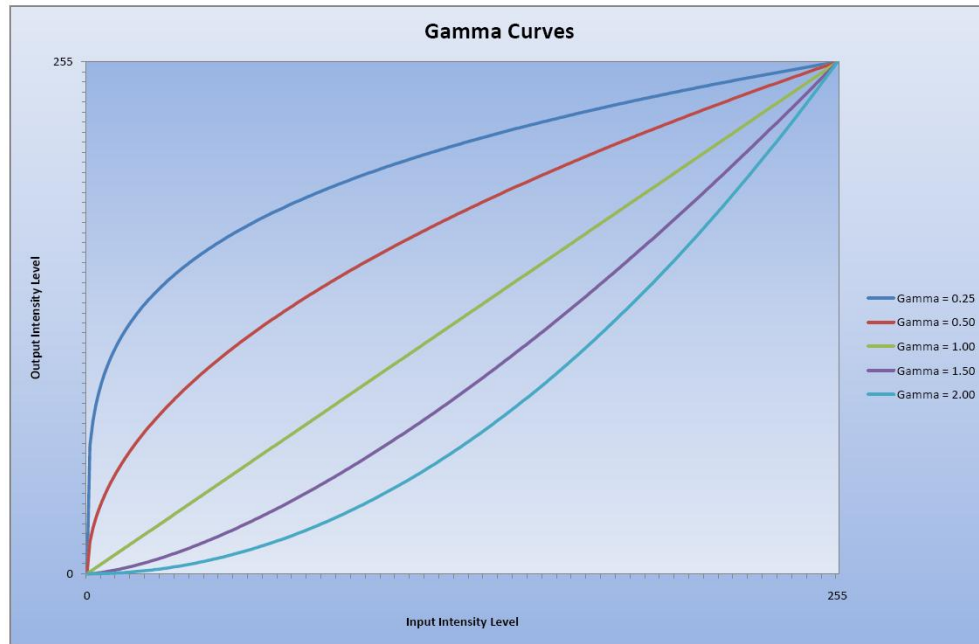
C2. Arithmetic and Logical Operations

a) Gamma Correction

Gamma (Power-Law Transformation), symbolized by the Greek letter γ , is described as the relationship between input and output. The input in question is the RGB intensity value of the image. The relationship between input and output in question is that the output is proportional to the input raised to the power of the Gamma value. The formula for Gamma Correction is as follows:

$$I' = 255 \times \left(\frac{I}{255} \right)^\gamma$$

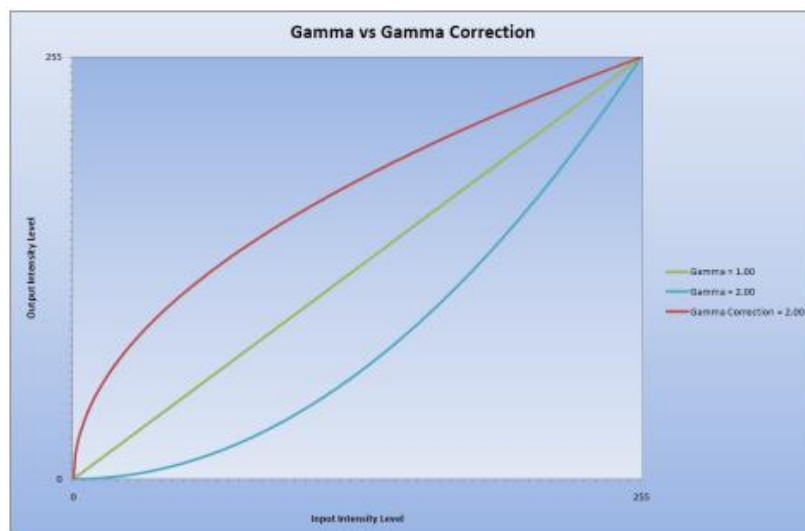
The following is an illustration of the output results of the Gamma curve at several Gamma values.



Note that with a Gamma value of 1, the input value is equal to the output value, indicated by a straight line. To calculate Gamma Correction, the input value is raised to the power of the inverse Gamma value. The formula for Gamma Correction is as follows:

$$I' = 255 \times \left(\frac{I}{255} \right)^{\frac{1}{\gamma}}$$

The following is a comparison graph between the gamma curve and the gamma correction curve:



The value used for Gamma depends on the application. Here are some examples of RGB images before and after processing with Gamma Correction.



Gamma Correction with Gamma value 0.5



Gamma Correction with Gamma value 6

b) Bit Depth

This operation is used to demonstrate image quantization. Quantization is performed on color depth values ranging from 1 to 7 bits. This transformation is a simulation only; the bits aren't actually 1-7 bits, just the color variations. Determine the depth level first, then the new color value is the result of the old color being operated on at the specified level. Here's the formula used:

$$level = \frac{255}{2^{bit_depth} - 1}$$

bit_dept is the desired color depth. The previous depth was 8 bits, or a gradient range from 0 to 255 (256 values). If you want a gradient range from 0 to 15 (16 values), select bit_dept = 4.

$$C' = round\left(\left(\frac{C}{level}\right) * level\right)$$



1bit depth gradation range from 0 to 1
(2 kinds)



2bit depth gradation range from 0 to 3 (4
kinds)



3bit depth gradation range from 0 to 7
(8 kinds)



4bit depth gradation range from 0 to 15
(16 kinds)



5bit depth gradation range from 0 to 31
(32 kinds)



6bit depth gradation range from 0 to 63
(64 kinds)



7bit depth gradation range from 0 to
127 (128 kinds)

c) PSNR

PSNR (Peak Signal-to-Noise Ratio) is the ratio between the maximum power value of an image and the power of the image affected by noise that affects the quality of the noisy image. To calculate the PSNR of an image, what needs to be done is to compare the denoised / compressed image with the ideal image that has maximum power (in this case, the original image).

PSNR is formulated with the following formula:

$$PSNR = 10 \log_{10} \left(\frac{(L-1)^2}{MSE} \right) = 20 \log_{10} \left(\frac{L-1}{RMSE} \right)$$

Where L is the number of possible color intensities of the image.

MSE is the mean squared error and is formulated as follows:

$$MSE = \frac{1}{MN} \sum_{i=0}^{m-1} \sum_{j=0}^{n-1} (Original(i,j) - Denoised(i,j))^2$$

RMSE (Root Mean-Square-Error) is the root of MSE .

The following is the function of PSNR in Python :

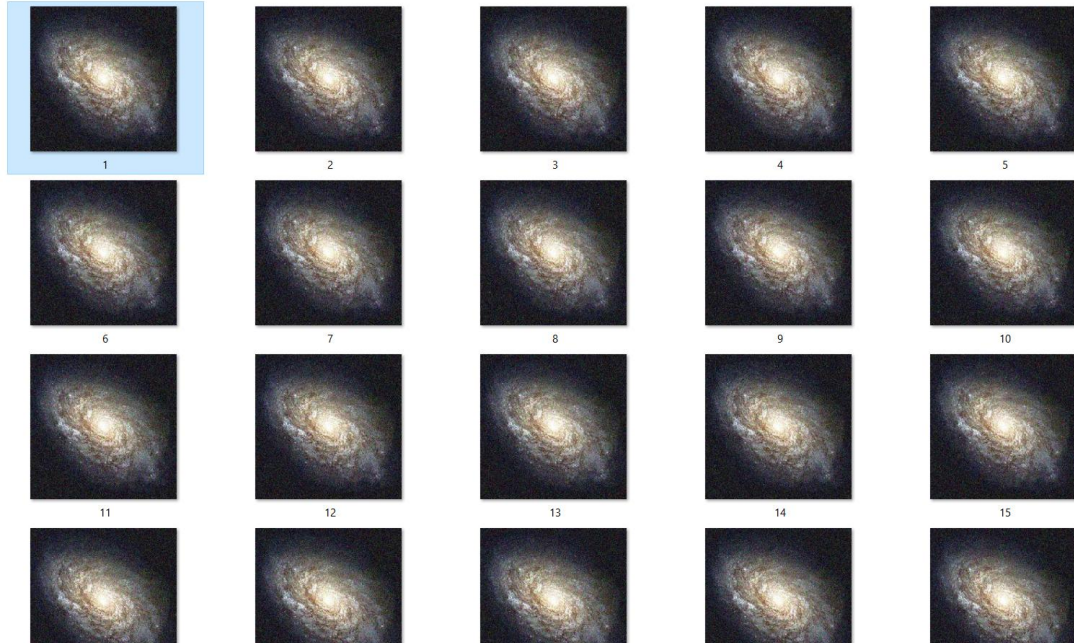
```
def PSNR(img1, img2):  
  
    mse = np.mean((img1 - img2) ** 2)  
  
    if(mse == 0): # MSE 0 maka tidak ada noise sama sekali, sehingga  
                  PSNR tidak memiliki arti  
  
        return 100  
  
    max_pixel = 255.0  
  
    psnr = 20 * log10(max_pixel / sqrt(mse))  
  
    return psnr
```

d) Average Denoising

Average denoising is an arithmetic operation applied to images for various useful purposes. This module will demonstrate the averaging operation for denoising/reducing noise in images. This operation calculates the average value of each pixel with the same coordinates in a set of images. This operation is typically performed to remove noise from external factors that occur during image acquisition. Average denoising is commonly used on images that contain external noise that arises from the distribution process from source to destination.

Image averaging works by calculating the average value of each pixel based on the number of images averaged. The more images averaged, the better the denoised result, the higher the PSNR value. Note that the higher the PSNR value, the better the quality of the denoised image.

The following image is intentionally given Gaussian noise randomly generated 100 times:



A collection of images that are randomly assigned Gaussian Noise



One of the images is enlarged and has been given Gaussian Noise

The following are the results of images that have been denoised using averaging on 5 images, 20 images, 50 images, and 100 images.



Average 5 images



Average 20 images



Average 50 images



Average 100 images

e) Image Masking

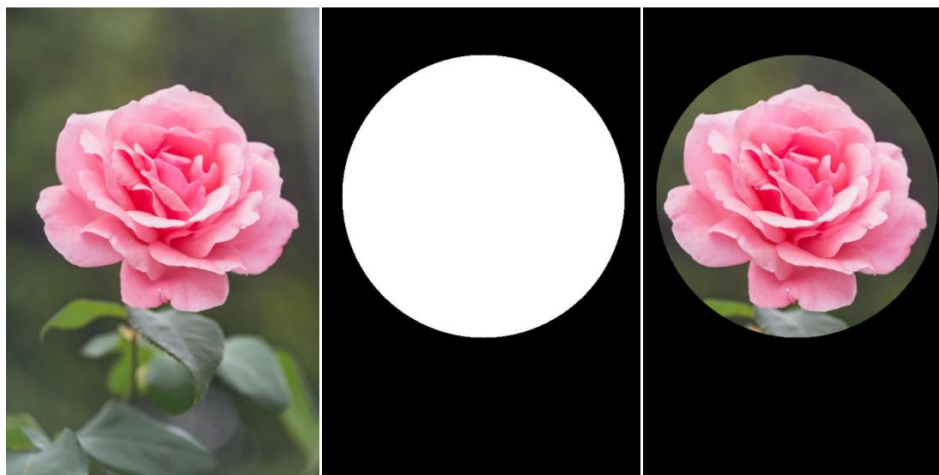
Image masking is an example of a logical operation used to process images. Refer to Gonzales's book, section 2.6.4, for a more detailed discussion of logical operations.

Some logical operators frequently used in image processing are the OR, AND, and NOT operators. In image masking, the AND operator is the most commonly used operator.

If it is assumed that two regions (sets) A and B consist of foreground pixels, then the OR operation of these two sets can contain values from A or B. The following figure illustrates Logical operations on images.



The following image shows the image results using the image masking operator AND.

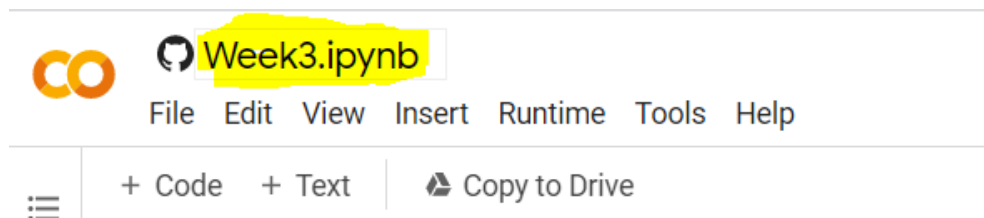


D. PRACTICAL

D1. Simple Image Operations

1. Open <https://colab.research.google.com/>, select the Github tab and make sure the selected repository is the same repository as the practicum in the first and second weeks.

Continue by creating a new notebook and rename the file to “Week3.ipynb”. **Note** : Don’t forget to save a copy to GitHub after making changes/when you’re done with the lab.



2. Access the images folder on your Google Drive with the following code:

```
[ ] from google.colab import drive  
  
drive.mount('/content/drive')  
  
Mounted at /content/drive
```

Follow the authorization flow until the message appears: “ *Mounted at /content/drive* ”.

3. Performing a linear transformation of brightness by inputting a specific constant value results in a color. As discussed in the theory review, the formula for performing a linear transformation of brightness is as follows:

$$g(x,y) = f(x,y) + b$$

where $g(x,y)$ is the pixel value after transformation, $f(x,y)$ is the original pixel value, and b is the brightness value.

Write the following code snippet to implement linear brightness on google colab:

▼ Transformasi Linier Brightness

Formula: $g(x,y)=f(x,y)+b$

$g(x,y)$ adalah nilai pixel setelah transformasi, $f(x,y)$ adalah nilai pixel asli, b adalah nilai brightness

```
print(' Mengubah tingkat kecerahan citra ')
print('-----')
try:
    brightness = int(input('Masukkan nilai kecerahan: '))
except ValueError:
    print('Error, not a number')

original = cv.imread('/content/drive/MyDrive/images/female.tiff')
brightness_image = np.zeros(original.shape, original.dtype)

#akses per piksel
for y in range(original.shape[0]):
    for x in range(original.shape[1]):
        for c in range(original.shape[2]):
            brightness_image[y,x,c] = np.clip(original[y,x,c] + brightness, 0, 255)

#cara simple tanpa for loop
#brightness_image = cv.convertScaleAbs(original, beta=brightness)

final_frame = cv.hconcat((original, brightness_image))
cv2.imshow('final_frame')
```

From the code snippet above, it can be seen that when run, the system will display a *text field* to enter the desired *brightness value constant*, and will be stored in the *brightness variable*. Then, the image on the drive that will be processed is determined and stored in the *original* variable. The next step is to access the input image pixels with 3 iterations. The first iteration is performed on *shape[0]* for the image height, the second iteration is performed on *shape[1]* for the image width, and the third iteration is performed on *shape[2]* which is the color channel in the image. After performing 3 iterations, a linear brightness transformation is performed by adding the brightness value to the input image. An example of the results of the program code above is as follows.

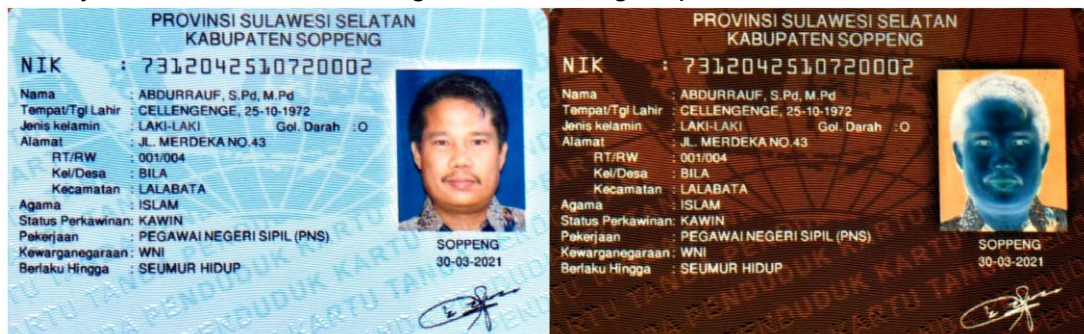
Mengubah tingkat kecerahan citra

Masukkan nilai kecerahan: 50



D1 PRACTICAL ASSIGNMENT

1. Implement image inversion on Google Collaboratory using the formulas found in the Theory Review section, resulting in the following output:



2. Implement the contrast transformation in Google Collaboratory using the formula found in the Theory Review section for contrast, resulting in the following output:

Mengubah kontras dan tingkat kecerahan citra

Masukkan tingkat kecerahan [-255 - 255]: 50

Masukkan kontras [1.0 - 3.0]: 2



3. Implement the logarithmic brightness transformation in Google Collaboratory using the formula found in the Theory Review section for log transformation, resulting in the following output:

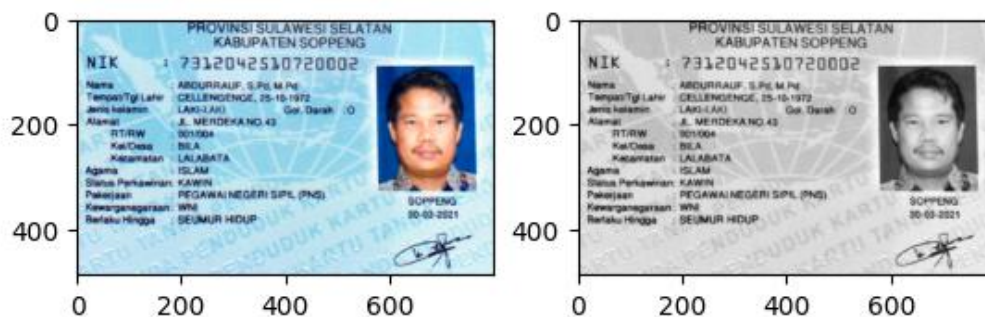
Mengubah tingkat kecerahan citra dengan Transformasi Log

Masukkan nilai kecerahan: 50



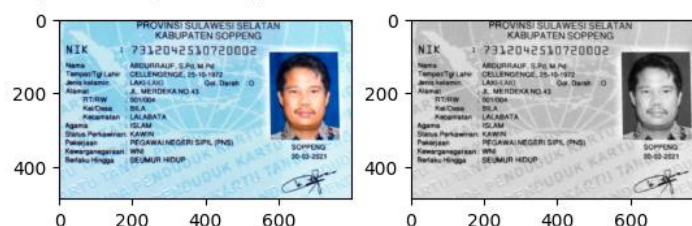
4. Implement the grayscale transformation using the averaging, lightness, and luminance methods on Google Collaboratory using the formulas found in the Theory Review section, resulting in the following output:

- a. Averaging



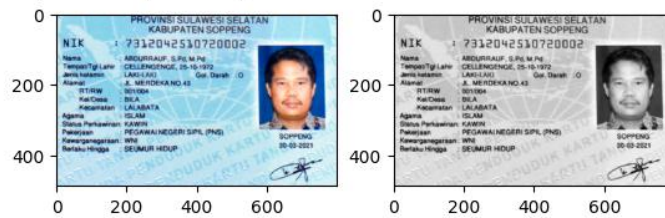
- b. Lightness

<matplotlib.image.AxesImage at 0x781d2577f7f0>

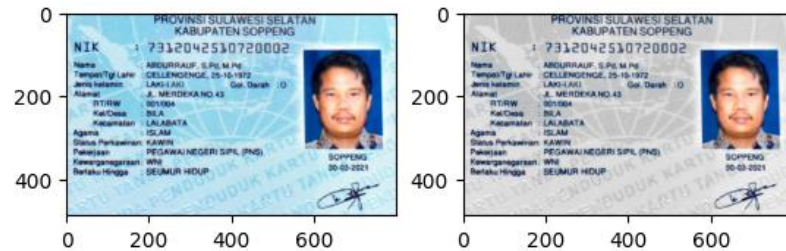


- c. Luminance

<matplotlib.image.AxesImage at 0x781d70a42fe0>



5. Display certain colors in an image and convert other colors to grayscale. For example, display blue in the input image and convert other non-blue parts to grayscale, as in the following example:



D2. Arithmetic and Logical Operations

1. Make Gamma Correction according to the following instructions

This experiment will ask you to create a gamma correction. In this experiment, the gamma value will be set by asking for user input. Below is the code to request the value. Continue the code by creating an image with gamma correction according to the given formula.

```
print(' Gamma Correction pada citra ')
print('-----')
try:
    gamma = int(input('Masukkan nilai Gamma: '))
except ValueError:
    print('Error, not a number')
```

Gamma Correction pada citra

Masukkan nilai Gamma: 3



The image above is an image processed using Gamma Correction with a gamma value of 3.

2. Create Image Depth Simulation

This experiment is used as a simulation of the image quantization process. In image quantization, pixels can be represented with n-bit depth (the default is 8-bit). In an 8-bit pixel, the possible colors are 256 colors, from 0 (0000 0000) to 255 (1111 1111). In a 7-bit pixel, the possible colors are 128 colors, from 0 (000 0000) to 127 (111 1111). The possible colors are obtained from the power of 2 of the number of bits. If 7 bits, then the number of colors is $2^7 = 128$, and so on.

The following is the code to read the input image and give the image depth value, please continue the following program code so that it produces output as in the example in the image.

```
bit_depth=2
level = 255 / (pow(2,bit_depth)-1)
original = cv.imread('/content/drive/MyDrive/Polinema/Kuliah/PCVK/
Images/female.tiff', cv.IMREAD_GRAYSCALE)
```



3. Create an Average Denoising module

Create an average denoising module according to the formula given in the previous sub-chapter.

The original image is provided at `/images/galaxy.jpg` .

100 Images with Gaussian Noise are provided at `/images/noises/*.jpg`

You can use the following code to read all images in one folder, use the glob module (`import glob`).

```
cv_img = []
for img in glob . glob( '/content/drive/MyDrive/Polinema/Kuliah/PCVK
/Images/noises/*.jpg' ):
    n = cv . imread(img)
    cv_img . append(n)
```

By using the code above, you just call the image in the folder using `cv_img[0]`, `cv_img[1]`, etc.

Record the PSNR results in the following table. Based on the results you have recorded, write your conclusions:

No	Number of Images in Average	Result Image	PSNR value (dB)
1.	10		
2 .	20		
3.	40		

4.	80		
5.	100		

From the PSNR results that you noted in the table above, the conclusion that can be drawn is:

.....

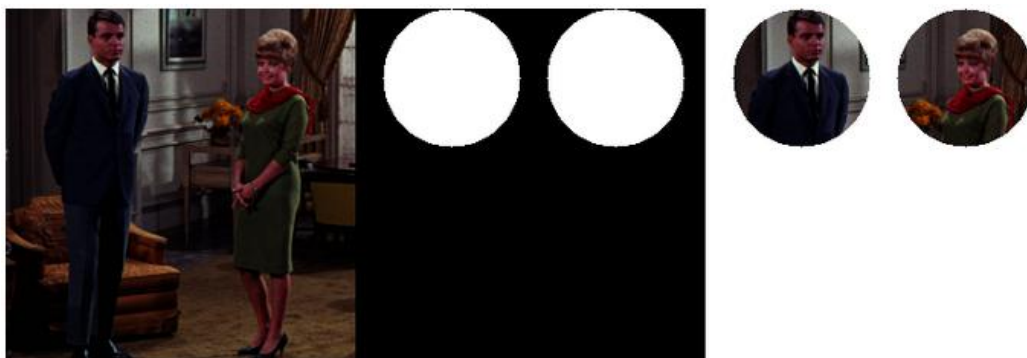
.....

.....

.....

4. Create image masking

Create an image mask for the following images. The left image is the original image (images/couple.tiff), while the rightmost image is the result:



Experiment with other operators and show the results in this module. Write down your analysis of why the output image looks like that.

No.	Operator	Image Input	Image Output
1.	NOT (complement)		



2.	OR (Or)		
3.	AND (And)		
4	NAND (Not And)		
5.	XOR (Exclusive Or)		

Write down the results of your analysis:

.....

.....

.....

.....

...

